

Sea-Bird Scientific Community Toolkit - CTD Processing

Introduction

This python notebook has been written to give Sea-Bird Scientific (SBS) customers and the scientific community at large a set of python code to be used to process and analyze data from the CTD family of SBS instruments. This toolkit was made in conjunction with the Fathom project. Great care has been taken to make sure that the python code that makes up this toolkit and its related packages gives the same results as the Fathom application.

Please contact [SBS customer support](#) for help or to request additional features.

Python Data Processing and Visualization

- [Low Pass Filter](#)
- [Align CTD](#)
- [Cell Thermal Mass](#)
- [Loop Edit](#)
- [Derive TEOS-10](#)
- [Wild Edit](#)
- [Window Filter](#)
- [Bin Average](#)
- [Buoyancy](#)

Init

The initialization code section below is used to import required libraries and to make a helper function that is used to make scatter charts with the Plotly package.

```
In [ ]: from pathlib import Path

# Third-party imports
import gsw
import numpy as np
import xarray as xr

# for static images
import plotly.io as pio
from IPython.display import Image, display

# Sea-Bird imports
```

```

import seabirdscientific.constants as const
import seabirdscientific.conversion as sc
import seabirdscientific.eos80_conversion as sc_eos80
import seabirdscientific.instrument_data as si
import seabirdscientific.processing as sp
import seabirdscientific.visualization as sv

def plot(
    dataset: xr.Dataset,
    title: str,
    x_names: list[str],
    x_bounds=None,
):
    if x_bounds is None:
        x_bounds = {0: [17, 24], 1: [-1, 6], 2: [-1, 6], 3: [-1, 6]}
    config = sv.ChartConfig(
        title=title,
        x_names=x_names,
        y_names=["pressure"],
        z_names=[],
        chart_type="overlay",
        bounds={
            "x": x_bounds,
            # 'y': { 0: [29, 32], 1: [29, 32], 2: [29, 32], 3: [29, 32] }
        },
        plot_loop_edit_flags=False,
        lift_pen_over_bad_data=True,
    )

    fig = sv.plot_xy_chart(dataset, config)
    fig["layout"]["yaxis"]["autorange"] = "reversed"
    fig.update_layout(height=800)
    # fig.show()
    # replace fig.show() with the following to generate static images
    filename = f"{title}.png"
    pio.write_image(fig, filename)
    display(Image(filename=filename))

# Load example cnv
ds = si.read_cnv_file(Path("example_data/19plus_V2/19plus_V2.cnv"), software="seaso

# Load example netcdf
# ds = xr.open_dataset("example_data/19plus_V2/19plus_V2.nc")

# rename variables for clarity in the rest of the notebook
ds = ds.rename_vars(
    {"tv290C": "temperature", "c0S/m": "conductivity", "prdM": "pressure", "sal00":
)

```

Low Pass Filter

This section shows how to use a low-pass filter on one or more columns of converted data. A low-pass

filter removes high frequency data to make the data smoother. To make sure there is zero phase shift (no time shift), the filter is first applied forward through the data and then backward through the forward-filtered data. This removes any time shifts caused by the filter.

Pressure data is typically filtered with a time constant equal to four times the CTD scan rate. Conductivity and temperature are typically filtered for *some* CTDs. Typical time constants are:

Instrument	Temperature	Conductivity	Pressure
SBE 19plus or 19plus V2	0.5	0.5	1.0

Filter Formula

For a low-pass filter with time constant Γ :

$$\Gamma = 1/2\pi f$$

$$T = \text{sample interval (seconds)}$$

$$S_0 = 1/\Gamma$$

Laplace transform of the transfer function of a low-pass filter (single pole) with a time constant of Γ seconds is:

$$H(s) = \frac{1}{1+(s/S_0)}$$

Using the bilinear transform:

$$S = f(z) \triangleq \frac{2(1-z^{-1})}{T(1+z^{-1})} = \frac{2(z-1)}{T(z+1)}$$

$$H(z) = \frac{1}{1+\frac{2(z-1)}{T(z+1)S_0}} = \frac{z^{-1}+1}{(1+\frac{2}{TS_0})(1+\frac{1-2/TS_0}{1+2/TS_0}z^{-1})}$$

If:

$$A = \frac{1}{1+\frac{2}{TS_0}}$$

$$B = \frac{1-\frac{2}{TS_0}}{1+\frac{2}{TS_0}}$$

Then:

$$H(z) = \frac{Y(z)}{X(z)} = \frac{A(z^{-1}+1)}{1+Bz^{-1}}$$

Where z^{-1} is the unit delay (one scan behind).

$$Y(z)(1+Bz^{-1}) = X(z)A(z^{-1}+1)$$

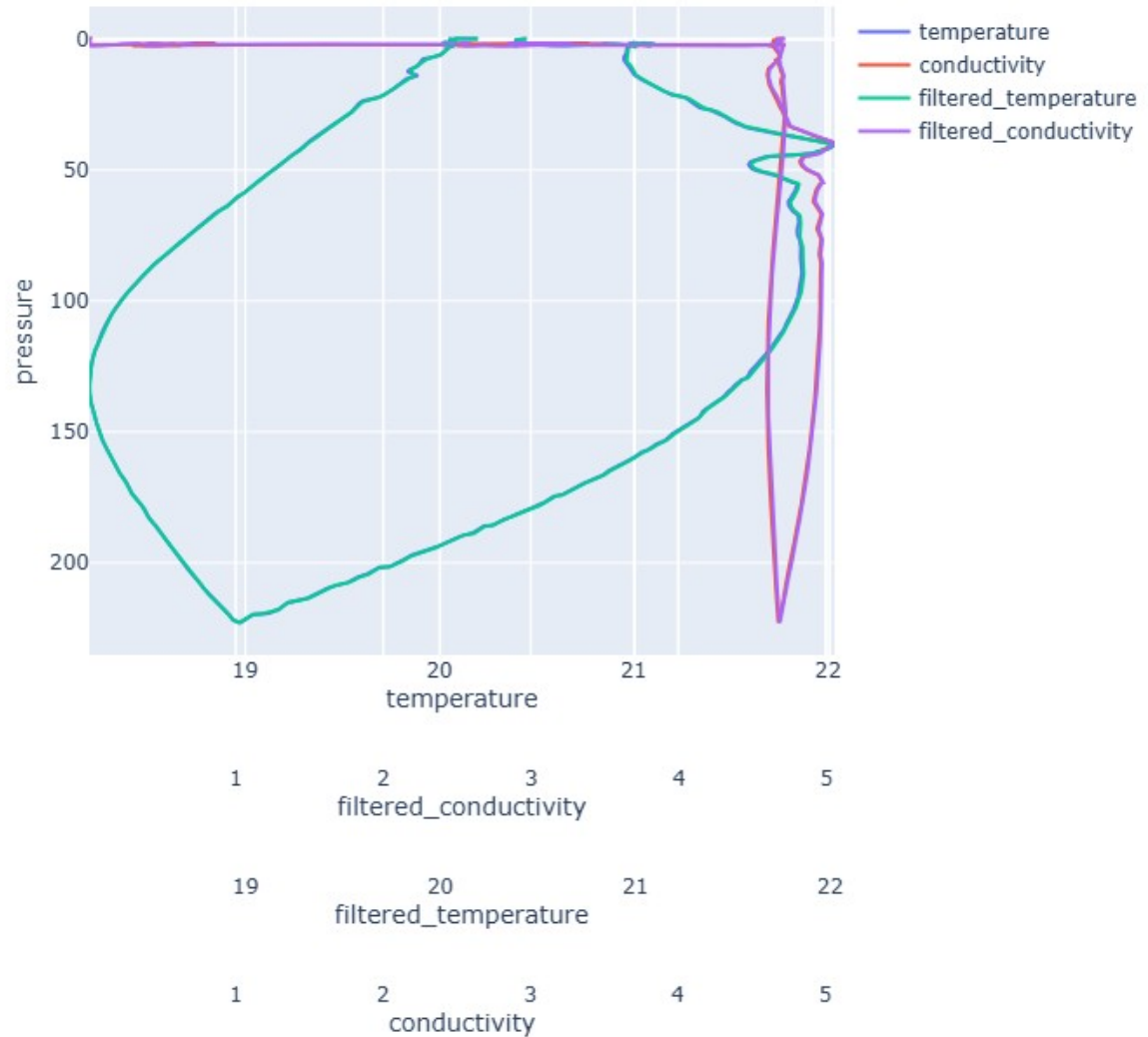
$$y[N] + By[N-1] = Ax[N-1] + Ax[N]$$

$$y[N] = A(x[N] + x[N - 1]) - By[N - 1]$$

```
In [2]: # Low pass filter
filtered_temperature = sp.low_pass_filter(
    x=ds["temperature"].values, time_constant=0.5, sample_interval=0.25
)
filtered_conductivity = sp.low_pass_filter(
    x=ds["conductivity"].values, time_constant=0.5, sample_interval=0.25
)
ds["filtered_temperature"] = ("scan", filtered_temperature)
ds["filtered_conductivity"] = ("scan", filtered_conductivity)

plot(
    dataset=ds,
    title="Low Pass Filter",
    x_names=["temperature", "conductivity", "filtered_temperature", "filtered_condu"],
    x_bounds={},
)
```

Low Pass Filter



Align CTD

This section shows how the Align CTD function aligns parameter data in time, relative to pressure, so that the calculated salinity, dissolved oxygen concentration, and other parameters are made with measurements from the same parcel of water. Typically, Align CTD is used to align temperature, conductivity, and oxygen measurements relative to pressure.

There are three principal causes of misalignment of CTD measurements:

- physical misalignment of the sensors in depth
- inherent time delay (time constants) of the sensor responses

- water transit time delay in the pumped plumbing line - the time it takes the parcel of water to go through the plumbing to each sensor (or, for free-flushing sensors, the related flushing delay, which depends on profiling speed)

When measurements are correctly aligned, salinity spikes (and density) errors are minimized, and oxygen data agrees with the correct pressure (e.g., temperature vs. oxygen plots agree between down and up profiles).

Conductivity and Temperature

Temperature and conductivity are often misaligned with respect to pressure. It can help to move the temperature and conductivity relative to pressure.

The best diagnostic of correct alignment is the removal of salinity spikes that align with very sharp temperature steps. To determine the best alignment, make a plot of 10 meters of temperature and salinity data at a depth that contains a very sharp temperature step. For the downcast, when temperature and salinity decrease with increased pressure:

- A negative salinity spike at the conductivity step means that conductivity leads temperature (conductivity sensor sees step before temperature sensor does). Move conductivity relative to temperature a negative number of seconds.
- If the salinity spike is positive, move conductivity relative to temperature a positive number of seconds.

The best alignment of conductivity and temperature occurs when the salinity spikes are minimized. It may be necessary to try different intervals to find the best alignment.

Typical Temperature Alignment

The SBE 19, 19plus, and 19plus V2 use a temperature sensor with a 0.5 second response time, which is slower than the conductivity sensor.

Instrument	Advance of temperature relative to pressure
19plus or 19plus V2	+0.5 seconds

Typical Conductivity Alignment

For an SBE 19plus or 19plus V2 with a standard 2000-rpm pump, do not move conductivity. However, if temperature is moved relative to pressure and you do not want to change the relative timing of temperature and conductivity, you must add the same interval to conductivity.

Oxygen

Oxygen data is also systematically delayed with respect to pressure. The two primary causes are the long time constant of the oxygen sensor (for the SBE 43, ranging from 2 seconds at 25 °C to approximately 5 seconds at 0 °C) and an additional delay from the transit time of water in the pumped plumbing line. As

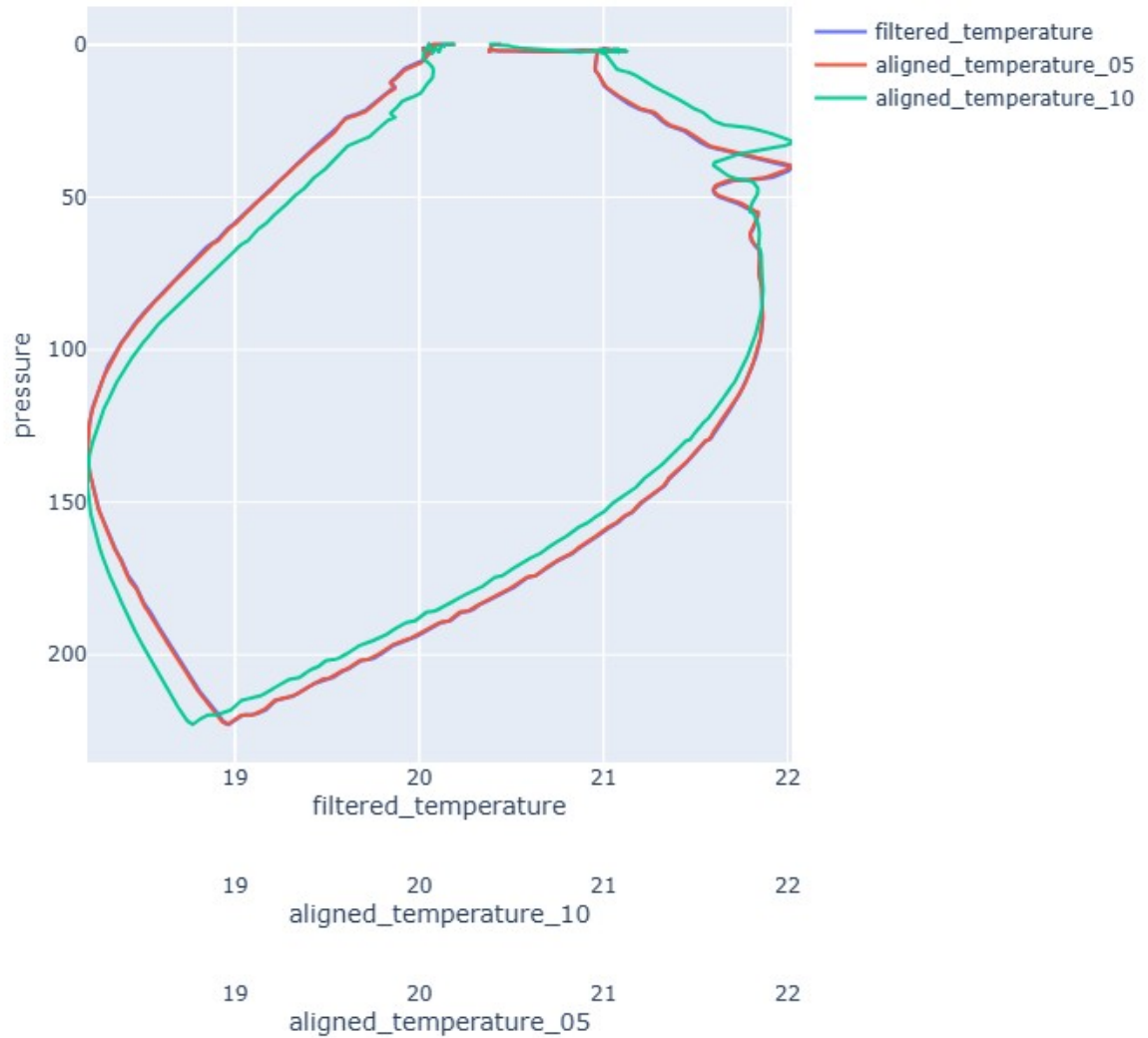
with temperature and conductivity, you can move oxygen data relative to pressure to adjust for this delay.

Instrument	Advance of oxygen relative to pressure
19plus or 19plus V2	+3 to +7 seconds

```
In [3]: aligned_temperature_05 = sp.align_ctd(
        x=ds["filtered_temperature"].values, offset=0.5, sample_interval=0.25
    )
    aligned_temperature_10 = sp.align_ctd(
        x=ds["filtered_temperature"].values, offset=10, sample_interval=0.25
    )
    ds["aligned_temperature_05"] = ("scan", aligned_temperature_05)
    ds["aligned_temperature_10"] = ("scan", aligned_temperature_10)

    plot(
        dataset=ds,
        title="Align CTD (shows multiple alignments for comparison purposes)",
        x_names=["filtered_temperature", "aligned_temperature_05", "aligned_temperature_10"],
        x_bounds={},
    )
```

Align CTD (shows multiple alignments for comparison purposes)



Cell Thermal Mass

This section shows the use of the Cell Thermal Mass function which uses a recursive filter to remove conductivity cell thermal mass effects from the measured conductivity. Typical values for alpha and 1/beta are:

Instrument	alpha	1/beta
SBE 19plus or 19plus V2 with TC duct and 2000 rpm pump	0.04	8.0

Cell Thermal Mass Formulas

$$a = \frac{2 \cdot \alpha}{(\text{sample interval} \cdot \beta + 2)}$$

$$b = 1 - \frac{2a}{\alpha}$$

$$\frac{dc}{dT} = 0.1 + 6e-4 * (\text{temperature} - 20)$$

$$dT = \text{temperature} - \text{previous temperature}$$

$$ctm = -b * \text{previous ctm} + a * \frac{dc}{dT} * dT$$

$$\text{corrected conductivity} = \text{conductivity} + ctm$$

Where sample interval is in seconds, temperature is in °C, ctm is in S/m, amplitude is the alpha value, and time_constant is 1/beta.

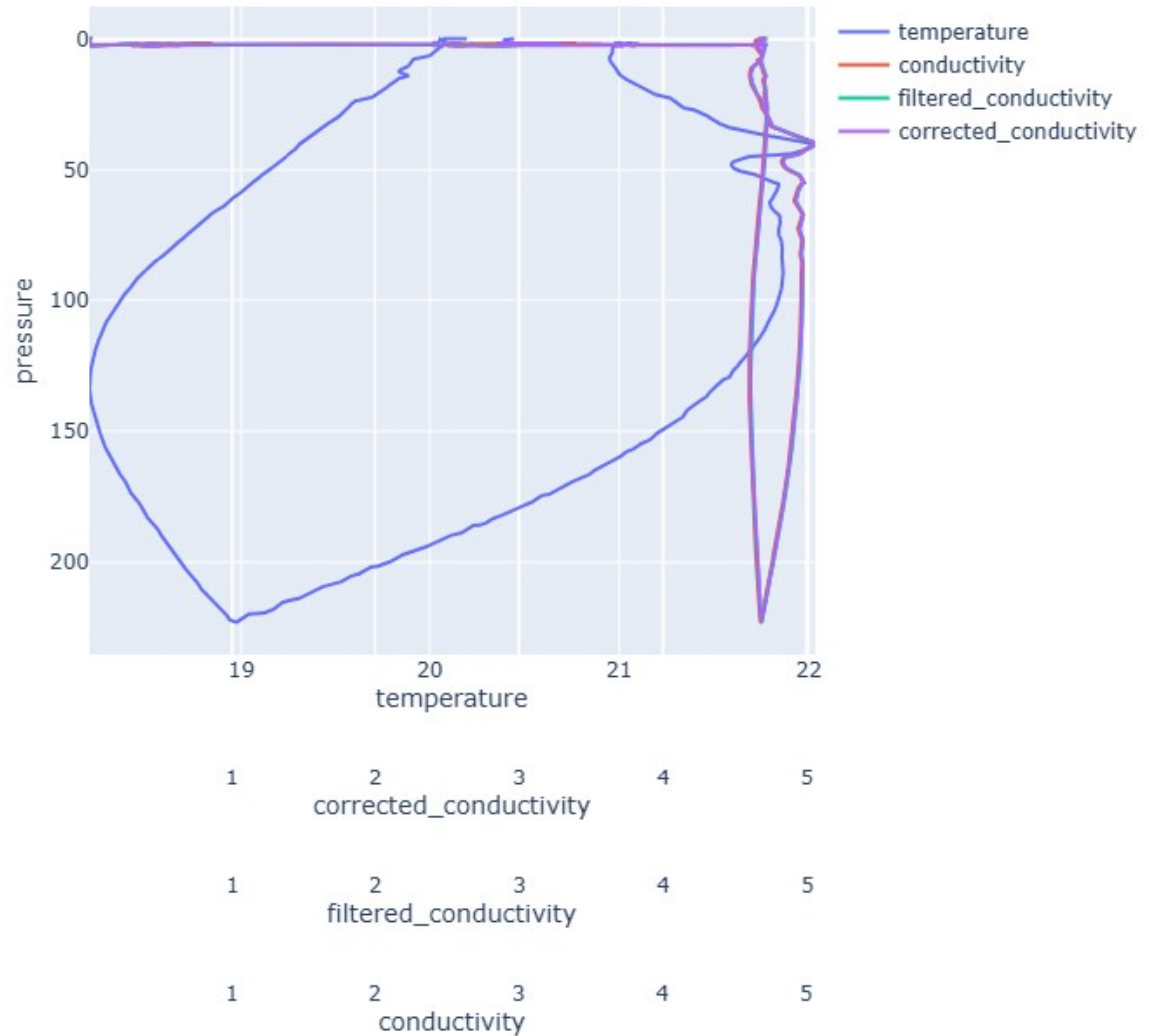
To determine the values for alpha and beta, see:

- [The Correction for Thermal-Lag Effects in Sea-Bird Scientific CTD Data](#), Morison, J., R. Andersen, Larson, N., D'Asaro, E., and Boyd, T., Journal of Atmospheric and Oceanic Technology (JAOT), V11(4), August 1994, 1151-1164
- [Thermal Inertia of Conductivity Cells: Theory](#), Lueck, R.G., Journal of Atmospheric and Oceanic Technology (JAOT), V7(5), Oct 1990, 741-755.

```
In [4]: corrected_conductivity = sp.cell_thermal_mass(
    temperature=ds["filtered_temperature"].values, # in deg. C
    conductivity=ds["filtered_conductivity"].values, # in S/m
    amplitude=0.04, # alpha
    time_constant=8, # 1/beta
    sample_interval=0.25, # in seconds
)
ds["corrected_conductivity"] = ("scan", corrected_conductivity)

plot(
    dataset=ds,
    title="Cell Thermal Mass",
    x_names=["temperature", "conductivity", "filtered_conductivity", "corrected_con",
    x_bounds={},
)
```

Cell Thermal Mass



Loop Edit

This section shows how the Loop Edit function identifies scans as bad. The flag value associated with the scan is set to badflag in input .cnv files that have pressure slowdowns or reversals (typically caused by ship heave). Optionally, Loop Edit can also identify scans related to an initial surface soak with badflag. The badflag value defaults to $-9.99\text{e-}29$.

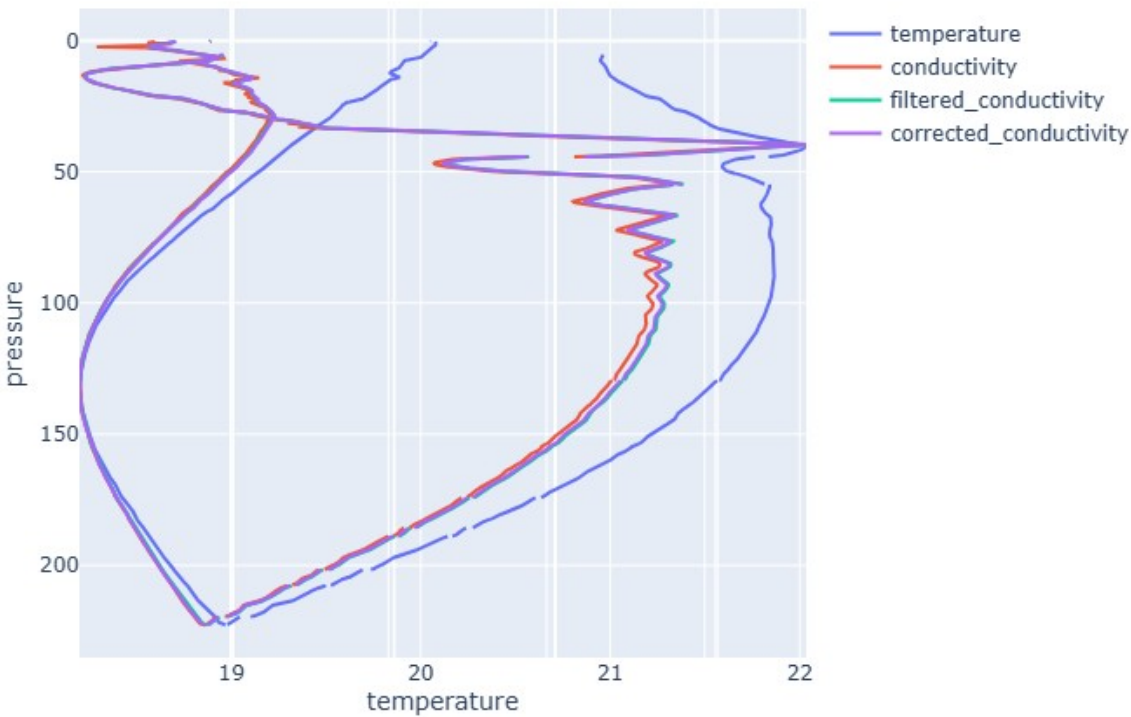
Loop Edit operates on three successive scans to determine velocity. This is such a fine scale that noise in the pressure channel from counting jitter or other unknown sources can cause Loop Edit to mark scans with badflag in error. Therefore, you must do the Filter on the pressure data to reduce noise before you

do the Loop Edit. See Filter for pressure filter recommendations for each instrument.

```
In [5]: if "flag" not in ds.data_vars:
        ds["flag"] = ("scan", np.zeros(len(ds["scan"])))

        ds["flag"].values = sp.loop_edit(
            measurand=ds["pressure"].values,
            flag=ds["flag"].values,
            sample_interval=0.25,
            min_velocity_type="fixed",
            min_velocity=0.1,
            use_deck_pressure_offset=False,
            units="pressure",
        )

        plot(
            dataset=ds,
            title="Loop Edit",
            x_names=["temperature", "conductivity", "filtered_conductivity", "corrected_conductivity"],
            x_bounds={},
        )
```



4.7	4.8	4.9	5
corrected_conductivity			
4.7	4.8	4.9	5
filtered_conductivity			
4.7	4.8	4.9	5
conductivity			

Derive TEOS-10

This section shows how the Derive TEOS-10 function uses temperature, conductivity, practical salinity, pressure, latitude, and/or longitude to compute the following thermodynamic parameters using TEOS-10 equations:

The table below references python functions from [TEOS-10](#) that are the same functions in SBE Data Processing.

Note: Results may differ slightly from SBE Data Processing due to GSW version differences.

Variable Name	GSW-Python function
Absolute Salinity	gsw.SA_from_SP
Absolute Salinity Anomaly	gsw.deltaSA_from_SP
adiabatic lapse rate	gsw.adiabatic_lapse_rate_from_CT
Conservative Temperature	gsw.CT_from_t
Conservative Temperature, freezing	gsw.CT_freezing
density, TEOS-10	gsw.rho ¹
dynamic enthalpy	gsw.dynamic_enthalpy
enthalpy	gsw.enthalpy
entropy	gsw.entropy_from_t
gravity	gsw.grav
internal energy	gsw.internal_energy
isentropic compressibility	gsw.kappa
latent heat of evaporation	gsw.latentheat_evap_CT
latent heat of melting	gsw.latentheat_melting
potential temperature	gsw.pt0_from_t
Preformed Salinity	gsw.Sstar_from_SA
Reference Salinity	gsw.SR_from_SP
saline contraction coefficient	gsw.beta
sound speed	gsw.sound_speed
specific volume	gsw.specvol
specific volume anomaly	gsw.specvol_anom_standard
temperature freezing	gsw.t_freezing
thermal expansion coefficient	gsw.alpha

¹ use gsw.rho with reference pressure for the sigmas

```
In [6]: absolute_salinity = gsw.SA_from_SP(SP=ds["salinity"].values, p=ds["pressure"].value
ds["absolute_salinity"] = ("scan", absolute_salinity)

conservative_temperature = gsw.CT_from_t(
    SA=ds["absolute_salinity"].values,
    t=ds["filtered_temperature"].values,
    p=ds["pressure"].values,
)
```

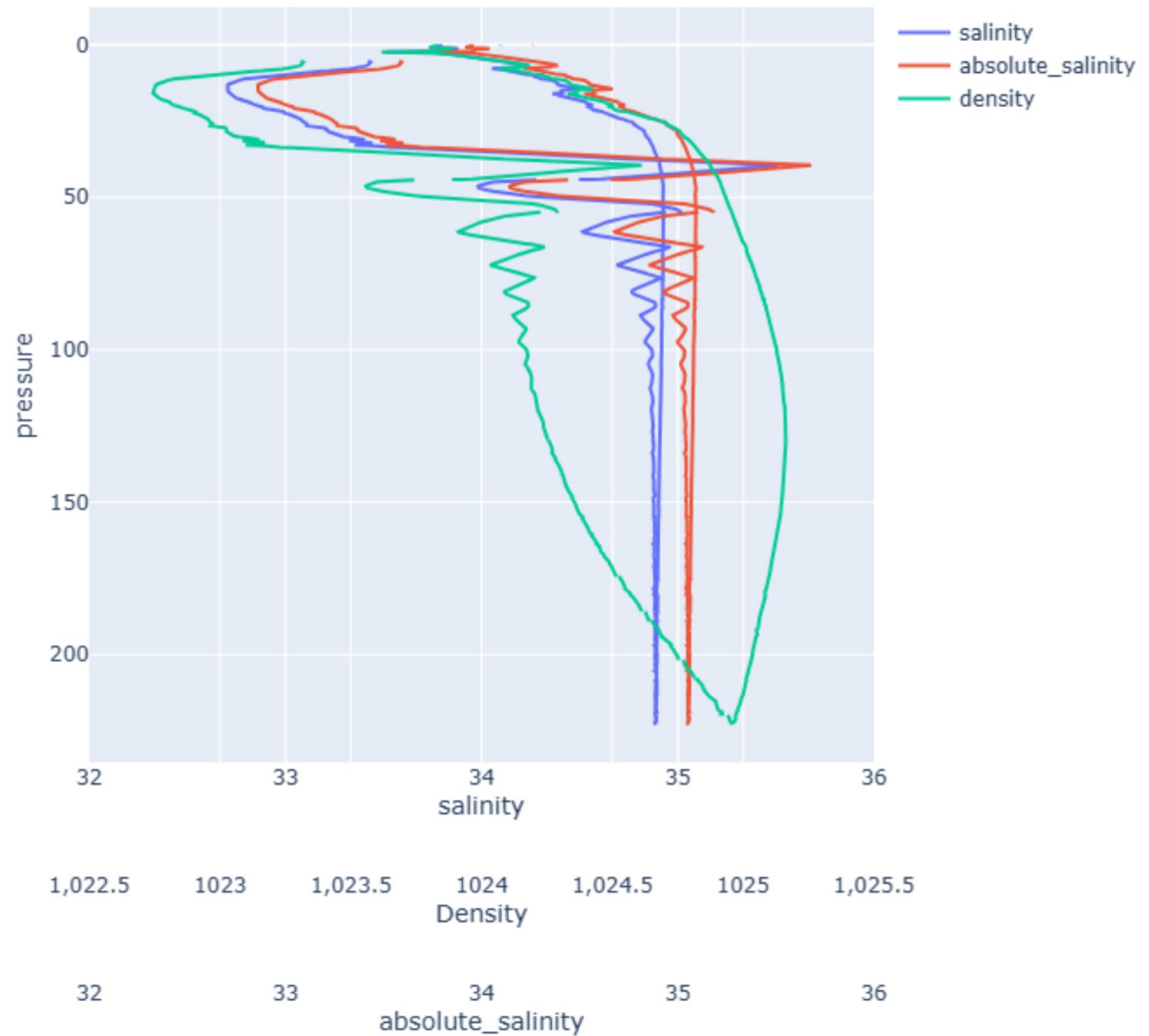
```
ds["conservative_temperature"] = ("scan", conservative_temperature)

density = gsw.rho(
    SA=ds["absolute_salinity"].values, CT=ds["conservative_temperature"].values, p=
)

ds["density"] = ("scan", density)

plot(
    dataset=ds,
    title="Derive TEOS-10",
    x_names=["salinity", "absolute_salinity", "density"],
    x_bounds={0: [32, 36], 1: [32, 36], 2: [1022.5, 1025.5]},
)
```

Derive TEOS-10



```
In [7]: potential_density_anomaly = gsw.sigma0(
        SA=ds["absolute_salinity"].values,
        CT=ds["conservative_temperature"].values,
    )

ds["potential_density_anomaly"] = ("scan", potential_density_anomaly)

sound_speed = gsw.sound_speed(
    SA=ds["absolute_salinity"].values,
    CT=ds["conservative_temperature"].values,
    p=ds["pressure"].values - const.SEA_LEVEL_PRESSURE_DBAR,
)

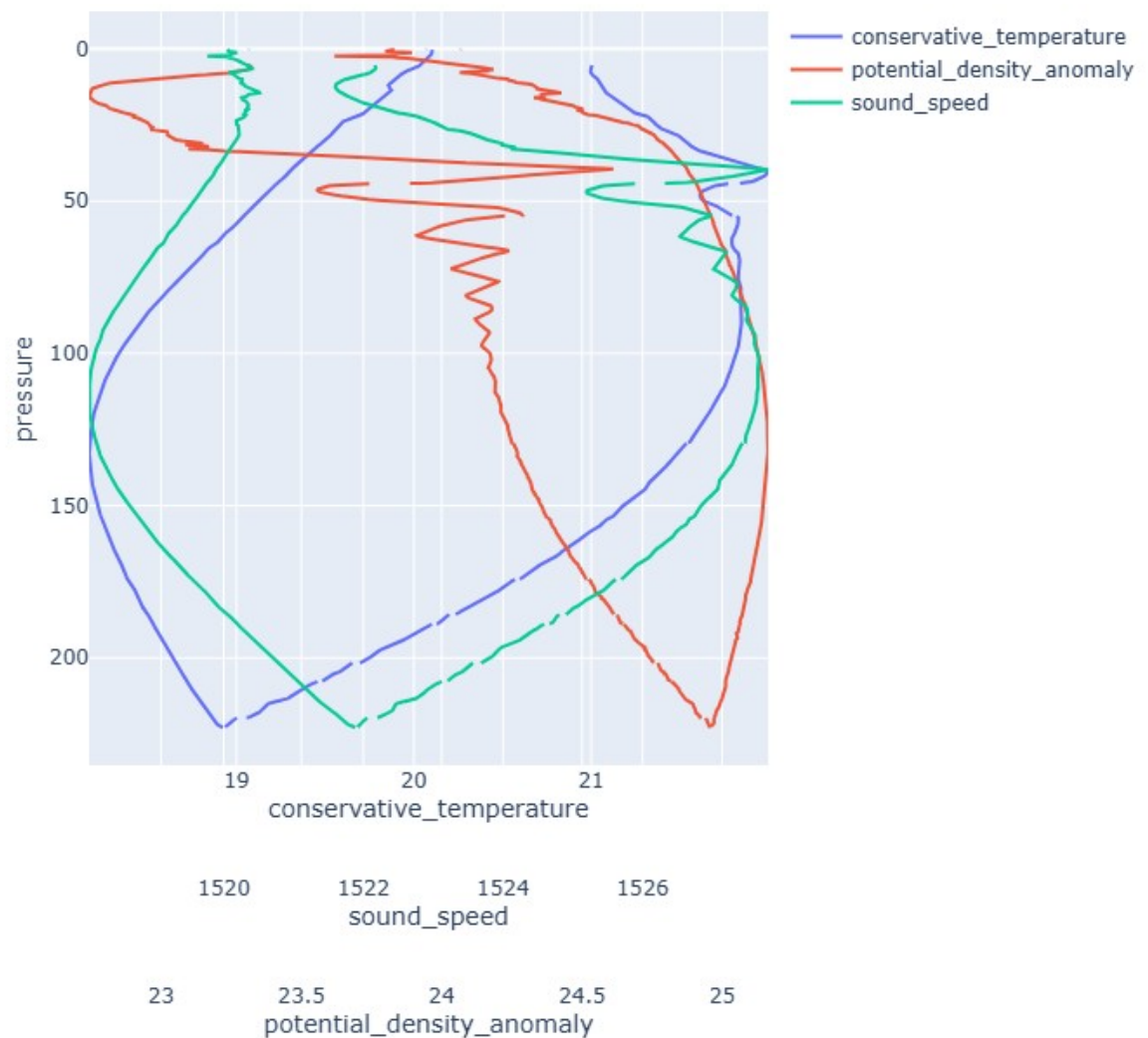
ds["sound_speed"] = ("scan", sound_speed)
```

```

plot(
  dataset=ds,
  title="Derive TEOS-10",
  x_names=[
    "conservative_temperature",
    "potential_density_anomaly",
    "sound_speed",
  ],
  x_bounds={},
)

```

Derive TEOS-10



This section shows how the Wild Edit function identifies wild points in the data. The data value is replaced with a badflag value, by default $-9.99\text{e-}29$. Wild Edit's algorithm requires two passes through the data. The first pass gives an accurate estimate of the data's true standard deviation, while the second pass replaces the appropriate data with badflag.

Wild Edit operates as follows:

1. Compute the mean and standard deviation of data in block (specified by Scans per Block) for each selected variable. Temporarily flag values whose distance from the mean is greater than or equal to `std_pass_1` standard deviations.
2. Recompute mean and standard deviation, and do not include temporarily flagged values. Identify the values whose distance from the mean is greater than `std_pass_2` standard deviations, then replace them with badflag.
3. Repeat Steps 1 and 2 for next block of scans. If the last block of data in the input file has less than specified number of scans, use data from previous block to fill in the block.

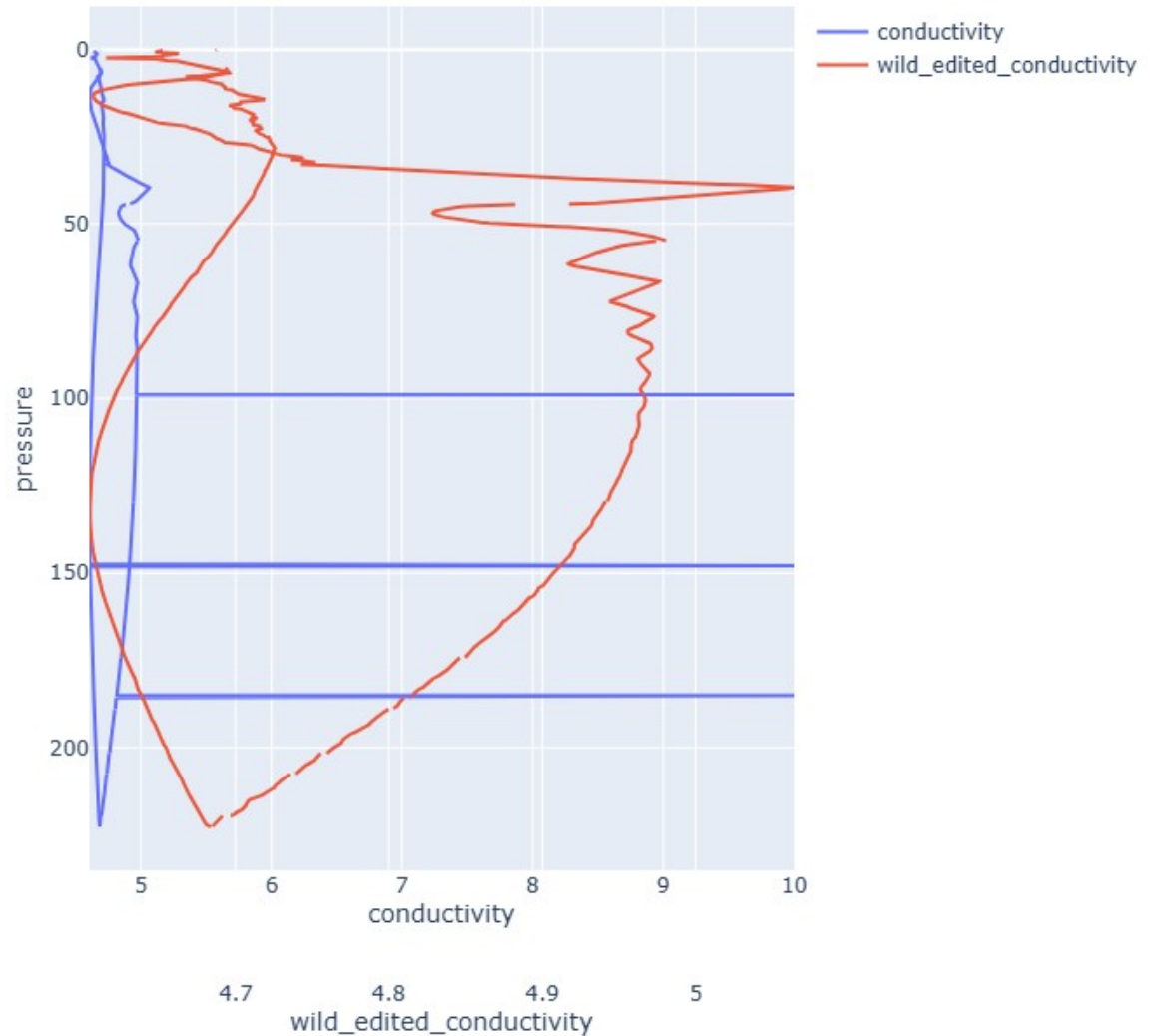
If the data file is particularly corrupted, you may need to do Wild Edit more than once, with different block sizes and number of standard deviations. If the input file has some variables with large values and some with relatively smaller values, it may be necessary to run Wild Edit more than once, and change the value for Keep data within this distance of mean so that it is meaningful for each variable. Increase Scans per block from 100 to approximately 500 to get better results.

```
In [8]: # mangle some data
ds["conductivity"].loc[{"scan": [1000, 1500, 2000]}] = 10

# remove the mangled data
wild_conductivity = sp.wild_edit(
    data=ds["conductivity"].values,
    flags=ds["flag"].values,
    std_pass_1=2,
    std_pass_2=20, # TODO: this breaks when set to 3 or lower
    scans_per_block=100,
    distance_to_mean=0,
    exclude_bad_flags=False,
)
ds["wild_edited_conductivity"] = ("scan", wild_conductivity)

plot(
    dataset=ds,
    title="Wild Edit",
    x_names=["conductivity", "wild_edited_conductivity"],
    x_bounds={},
)
```

Wild Edit



Window Filter

The Window Filter function gives four types of window filters and a median filter that can be used to make the data smoother:

- Window filters calculate a weighted average of data values about a center point and replace the data value at the center point with this average.
- The median filter calculates a median for data values around a center point and replaces the data value at the center point with the median

Descriptions and Formulas

Shape and length define filter windows:

- Window Filter gives four window shapes: boxcar, cosine, triangle, and Gaussian.
- The minimum window length is 1 scan, and the maximum is 511 scans. Window length must be an odd number, so that the window has a center point. If a window length is specified as an even number, Window Filter automatically adds 1 to make the length odd.

The window filter calculates a weighted average of data values around a center point, using the transfer function below:

$$y(n) = \sum_{k=-(L-1)/2}^{(L-1)/2} w(k)x(n-k)$$

The window filter process is similar for all filter types:

1. Filter weights are calculated (see the equations below).
2. Filter weights are normalized to sum to 1.
 - When the data includes scans that have been identified as bad, and the `exclude_flags` variable is set to true, the weights are renormalized, and do not include the filter elements that would operate on the bad data point.

In the equations below:

L = window length in scans (must be odd)

n = window index, $-\frac{L-1}{2} \dots \frac{L-1}{2}$

$w(n)$ = set of window weights

Boxcar Filter

$$w(n) = \frac{1}{L}$$

Cosine Filter

$$w(n) = \cos\left(\frac{n\pi}{L+1}\right)$$

Triangle Filter

$$w(n) = 1 - \frac{|2n|}{L+1}$$

Gaussian Filter

$$phase = \frac{offset}{sample\ interval}$$

$$scale = \log(2) \left(2 \frac{sample\ interval}{half\ width\ (scans)} \right)^2$$

$$w(n) = e^{-(n-phase)^2*scale}$$

The Gaussian window has parameters of halfwidth (in scans) and offset (in time), in addition to window length (in scans). These extra parameters filter and move data in time in one operation. Halfwidth determines the width of the Gaussian curve. A window length of 9 and halfwidth of 4 gives a set of filter weights that fills the window. A window length of 17 and halfwidth of 4 gives a set of filter weights that fills only half the window. If the filter weights do not fill the window, the offset parameter may be used to move the weights within the window so that the edge of the Gaussian curve is not cut.

Median Filter

The median filter does not make data smoother in the same sense as the window filters described above. The median filter is most useful in spike removal. A median value is determined for a specified window, and the data value at the window's center point is replaced by the median value.

```
In [ ]: boxcar_temperature = sp.window_filter(
        data_in=ds["temperature"].values,
        flags=ds["flag"].values,
        window_type="boxcar",
        window_width=21,
        sample_interval=0.25,
    )
    ds["boxcar_temperature"] = ("scan", boxcar_temperature)

    plot(
        dataset=ds,
        title="Boxcar Window Filter",
        x_names=["temperature", "boxcar_temperature"],
        x_bounds={},
    )
```

Bin Average

The Bin Average function is used to reduce the number of data points in a dataset by averaging data values within specified pressure, depth, time, or scan number bins.

Descriptions and Formulas

The center value of the first (not surface) bin is set equal to the bin size. The surface bin, if included, cannot overlap the first bin. For example, if the bin size is 10 dbar, the center of the first bin is at 10 dbar, and the bin extends from 5 to 15 dbar. If a surface bin is included, it could have a maximum bin size of 5 dbar, extending from 0 to 5 dbar.

The algorithms used for each type of averaging are as follows:

Pressure or Depth Bins (no interpolation)

1. Add together valid data for scans with $\text{BinMin} < \text{pressure} \leq \text{BinMax}$ for upcast, $\text{BinMin} \leq \text{pressure}$

- < BinMax for downcast.
- 2. Divide sum by the number of valid data points to obtain average, and write average to output dataframe.
- 3. Repeat Steps 1 through 2 for each variable.
- 4. For next bin, compute center value and repeat Steps 1 through 3.

Pressure or Depth Bins (with interpolation)

1. Add together valid data for scans with BinMin < pressure <= BinMax for upcast, BinMin <= pressure < BinMax for downcast.
2. Divide sum by the number of valid data points to obtain average, and write average to output dataframe.
3. Interpolate as follows, and write interpolated value to output file:

$$P_p = \text{average pressure of previous bin}$$

$$X_p = \text{average value of variable in previous bin}$$

$$P_c = \text{average pressure of current bin}$$

$$X_c = \text{average value of variable in current bin}$$

$$P_i = \text{center value for pressure in current bin}$$

$$X_i = \text{interpolated value of variable}$$

$$= \frac{(X_c - X_p) * (P_i - P_p)}{(P_c - P_p)} + X_p$$
4. Repeat Steps 1 through 3 for each variable.
5. Compute center value and Repeat Steps 1 through 4 for next bin. Values for first bin are interpolated after averages for second bin are calculated; values from next (second) bin instead of previous bin are used in equations.

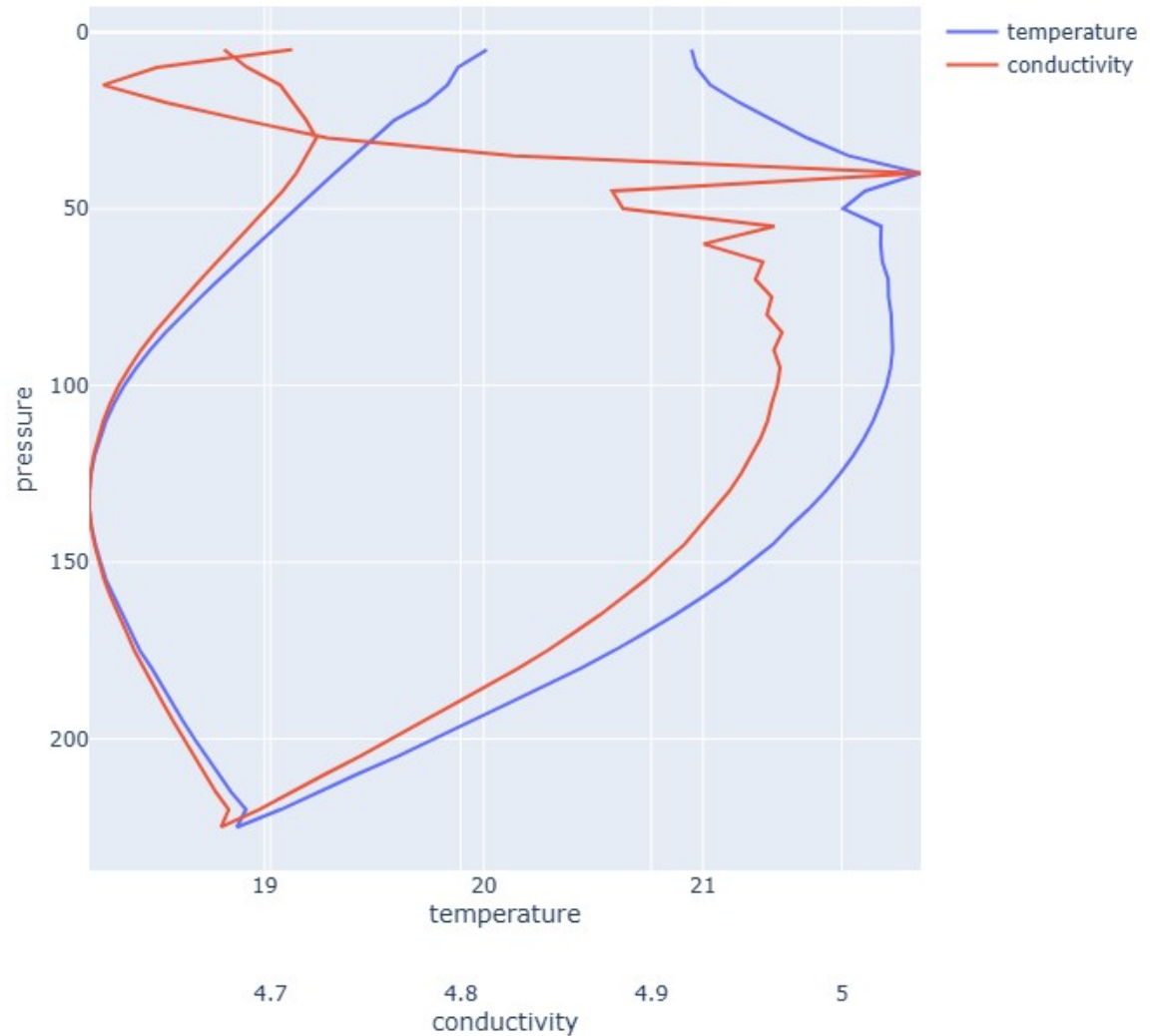
Scan Number or Time bins

Scan number or time bin processing is similar to processing pressure bins without interpolation.

```
In [9]: binned_ds = sp.bin_average(
        dataset=ds,
        bin_variable="pressure",
        bin_size=5,
        interpolate=True,
        include_scan_count=False,
    )

    plot(
        dataset=binned_ds,
        title="Bin Average",
        x_names=["temperature", "conductivity"],
        x_bounds={},
    )
```

Bin Average



Buoyancy

There are two Buoyancy functions, a modern formula that uses TEOS-10 variables and a legacy (Brunt-Väisälä) formula that calculates buoyancy frequency (N) and stability (E) using the Fofonoff adiabatic leveling method. The Buoyancy formula can only be used on data that has been binned with Bin Average.

Descriptions and Formulas

Calculate buoyancy variables for pressure values centered in window. Buoyancy converts window size from decibars to scans based on pressure interval between scans in input file. If window size is less than 3

scans, Buoyancy sets it to 3 scans. If window size is an even number of scans, Buoyancy adds 1 scan to window size.

The relationship between frequency N and stability E is:

$$N^2 = gE[\text{rad}^2/\text{s}^2]$$

Where g = gravity [m / s^2]

Algorithm 1 - Modern formula

1. Compute averages:

p_{bar} = average pressure in buoyancy window [decibars]

t_{bar} = average conservative temperature in buoyancy window [deg C]

s_{bar} = average absolute salinity in buoyancy window [g/kg]

2. Compute specific volume, thermal expansion coefficient (α), and saline contraction coefficient (β)

$$\text{specvol} = \text{gsw.specvol}(s_{bar}, t_{bar}, p_{bar}) [\text{m}^3/\text{kg}]$$

$$\alpha = \text{gsw.alpha}(s_{bar}, t_{bar}, p_{bar}) [1/\text{K}]$$

$$\beta = \text{gsw.beta}(s_{bar}, t_{bar}, p_{bar}) [\text{kg/g}]$$

3. Compute the gradients dt/dp and ds/dp

Use least squares fit to compute the linear gradient of conservative temperature with respect to pressure dt/dp in the buoyancy window.

Use least squares fit to compute the linear gradient of absolute salinity with respect to pressure ds/dp in the buoyancy window.

4. Compute N^2 , N , E , and $10^{-8}E$:

$$N^2 = \frac{g^2}{\text{specvol} \times 1e4} * (\beta \frac{ds}{dp} - \alpha \frac{dt}{dp}) [\text{rad}^2/\text{s}^2]$$

$$N = \frac{3600}{2\pi} \text{sgn}(N^2) \sqrt{N^2} [\text{cycles}/\text{hour}]$$

$$E = \frac{N^2}{g} [\text{rad}^2/\text{m}]$$

$$E = 10^8 \frac{N^2}{g} [10^{-8} \text{rad}^2/\text{m}]$$

Algorithm 2 - Brunt-Väisälä formula

1. Compute averages:

p_{bar} = average pressure in buoyancy window [decibars]

t_{bar} = average temperature in buoyancy window [deg C]

s_{bar} = average salinity in buoyancy window [PSU]

$$\rho_{bar} = \text{density}(s_{bar}, t_{bar}, p_{bar}) - 1000 \text{ [Kg / m}^3\text{]}$$

2. Compute the vertical gradient:

$$\theta = \text{potential temperature}(s, t, p, p_{bar})$$

$$v = 1 / (\text{density}(s, \theta, p_{bar}) - 1000)$$

Where s , t , and p are the averaged values for salinity, temperature, and pressure calculated in Bin Average.

Use least squares fit to compute the linear gradient dv/dp in the buoyancy window.

3. Compute N^2 , N , E , and $10^{-8}E$:

$$N^2 = -1.0 e^{-4} \rho_{bar}^2 g^2 \frac{dv}{dp} \text{ [rad}^2/\text{s}^2\text{]}$$

$$N = \frac{3600}{2\pi} \text{sgn}(N^2) \sqrt{N^2} \text{ [cycles/hour]}$$

$$E = \frac{N^2}{g} \text{ [rad}^2/\text{m]}$$

$$E = 10^8 \frac{N^2}{g} \text{ [10}^{-8}\text{rad}^2/\text{m]}$$

```
In [12]: # get modern buoyancy
(buoyancy_frequency_squared, buoyancy_frequency, stability, scaled_stability) = sc.
    temperature=binned_ds["temperature"],
    salinity=binned_ds["salinity"],
    pressure=binned_ds["pressure"],
    latitude=[0],
    longitude=[0],
    window_size=25,
)
binned_ds["N2"] = ("bin_number", buoyancy_frequency_squared)
binned_ds["N"] = ("bin_number", buoyancy_frequency)
binned_ds["E"] = ("bin_number", stability)
binned_ds["E10"] = ("bin_number", scaled_stability)

# get EOS80 buoyancy
(
    buoyancy_frequency_squared_eos80,
    buoyancy_frequency_eos80,
    stability_eos80,
    scaled_stability_eos80,
) = sc_eos80.buoyancy_eos80(
    temperature=binned_ds["temperature"],
    salinity=binned_ds["salinity"],
    pressure=binned_ds["pressure"],
    latitude=[0],
    longitude=[0],
    window_size=25,
)
binned_ds["N2_EOS80"] = ("bin_number", buoyancy_frequency_squared_eos80)
```

```

binned_ds["N_EOS80"] = ("bin_number", buoyancy_frequency_eos80)
binned_ds["E_EOS80"] = ("bin_number", stability_eos80)
binned_ds["E10_EOS80"] = ("bin_number", scaled_stability_eos80)

plot(
  dataset=binned_ds,
  title="Buoyancy",
  x_names=["N2", "N2_EOS80"],
  x_bounds={},
)

```

Buoyancy

